

LABORATORY 03 – Base Platform and Application Layer Protocols

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes



COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

February 23, 2025

OBJETIVE

For this laboratory, we want to continue learning the installation of base software, focusing on DNS and NTP services, as well as Shell programming to automate its configuration and management, with the aim of improving the administration and efficiency of computer systems.

MATERIALS & TOOLS

- Laboratory Computers
- Internet Access
- Virtualization Software

INTRODUCTION

Nowadays in any IT environment, correctly configuring essential services such as DNS and NTP is key to ensuring efficient network operation. DNS (Domain Name System) is a fundamental protocol that translates domain names into IP addresses, allowing devices to communicate without having to manage numerical addresses. On the other hand, NTP ensures time synchronization on all connected computers, avoiding discrepancies that could affect security, event logs, and the execution of critical services.

The implementation of these protocols in virtualized environments is increasingly common, allowing greater flexibility in the administration of servers and networks. Through virtualization, it is possible to configure and test complete infrastructures without the need for additional hardware, facilitating the replication of real environments in test laboratories.

In this lab, we will focus on the installation and configuration of DNS and NTP servers in a virtualized environment, guaranteeing their correct operation within the network. Additionally, the use of Shell scripts will be explored to automate repetitive tasks, optimizing the management and configuration of these services.

THEORETICAL FRAMEWORK

The **Domain Name System (DNS)** is a hierarchical and decentralized naming system used to resolve domain names into IP addresses. It enables user-friendly domain names to be translated into machine-readable addresses, allowing seamless internet navigation. The DNS operates through different types of records, including:

- **A (Address) Record:** Maps a domain name to an IPv4 address.
- **AAAA Record:** Similar to A records but maps to an IPv6 address.
- **NS (Name Server) Record:** Specifies the authoritative DNS servers for a domain.
- **CNAME (Canonical Name) Record:** Creates an alias for another domain name.
- **MX (Mail Exchange) Record:** Defines mail servers responsible for email delivery.

DNS servers can operate in different configurations: **primary** (*master*) servers store original zone files, while **secondary** (*slave*) servers synchronize from the primary server for redundancy and load distribution.

BIND (*Berkeley Internet Name Domain*) is one of the most widely used DNS server software solutions across Unix-like systems and Windows Server. It provides a flexible and robust implementation of the DNS protocol, supporting both authoritative and recursive name resolution. BIND allows administrators to configure primary and secondary DNS servers, manage forward and reverse lookup zones, and implement security features such as DNSSEC. It is highly customizable through the `named.conf` configuration file, where administrators define zones, caching policies, and access controls. Due to its cross-platform compatibility, BIND remains a preferred choice for enterprises and ISPs requiring reliable domain name resolution services.

Additionally, **Shell scripting** is a crucial automation tool in Unix-like systems, allowing users to execute commands, schedule tasks, and manage system processes efficiently. Shell scripts can handle tasks such as process monitoring, system resource management, and scheduled execution using tools like *cron* and *at*.

By combining DNS configuration with Shell scripting, administrators can automate critical network services, ensuring efficient and reliable infrastructure management.

BASE SOFTWARE

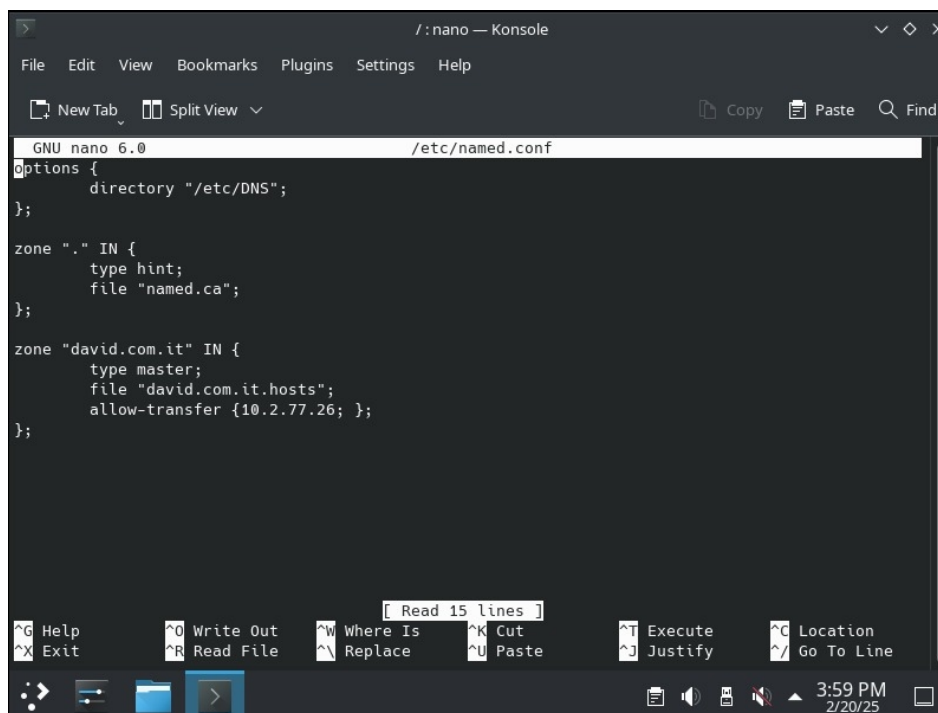
This section is practically all the laboratory work. Well, in this, the different configurations and scripts that we will use will be made to understand the operation of the target protocols by setting up several of these services.

LINUX DNS SERVER - BIND

To begin, we are going to set up DNS servers on our virtual machines using the BIND package. This allows us to use the DNS protocol to connect the devices and be able to test them. To do this, we will have to install or update the package on the machines; in our case, Slackware already had it installed as did Windows, for NetBSD we did have to install it.

Once we confirm that BIND is in our system, we have to enter our package configuration to place the servers that we will have. In our case, in BSD, we had to create it because it did not come by default. Next, we are going to place our servers on each machine. But first, we define that Slackware will be the main DNS, so in the configuration we will place this as **master**. In the case of BSD, it would be placed in **slave** mode. In this way, we can differentiate which is the main server and which is secondary. For Windows, we explain later, but basically it will be in **slave** mode by default.

- Slackware:



```
/: nano — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find
GNU nano 6.0 /etc/named.conf
options {
    directory "/etc/DNS";
};

zone "." IN {
    type hint;
    file "named.ca";
};

zone "david.com.it" IN {
    type master;
    file "david.com.it.hosts";
    allow-transfer {10.2.77.26; };
};

[ Read 15 lines ]
^G Help      ^O Write Out  ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File  ^_ Replace   ^U Paste     ^J Justify
^C Location
^_ Go To Line
3:59 PM
2/20/25
```

- NetBSD:

```

GNU nano 8.3 /etc/named.conf
options {
    #directory "/etc/namedb";
    directory "/etc/DNS";
    allow-recursion { localhost; localnets; };
};

zone "andres.com.it" {
    type slave;
    file "andres.com.it.hosts";
    masters {10.2.77.28; };
};

# example secondary server config:
#
# zone "Berkeley.EDU" {
#     type slave;
#     file "berkeley.edu.cache";
#     masters {

```

[Read 46 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line

Now we need to designate the root servers and canonical names so that we can have access to the network. To do this, we go to the configuration of each server that we created, and that is identified with the previous names and we put them. As shown in the two next images.

```

/: nano — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find
GNU nano 6.0 /etc/DNS/david.com.it.hosts
$TTL 86400
@ IN SOA ns1.david.com.it. root.david.com.it. (
    2024022001 ; Serial
    3600      ; Refresh
    1800      ; Retry
    604800    ; Expire
    86400     ; Minimum TTL
)

@ IN NS ns1.david.com.it.

ns1 IN A 10.2.77.28

@ IN A 10.2.77.28 ; IP for david.com.it
netBSD IN A 10.2.77.26 ; IP for andres.org.uk

samba IN CNAME netBSD.david.com.it

```

[Read 16 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line

3:59 PM 2/20/25

The terminal window displays the contents of the file `/var/named/named.ca` using `nano`. The file contains the following DNS zone data:

```

file /domain/named.cache
on server FTP.INTERNIC.NET
-OR- RS.INTERNIC.NET

last update: January 29, 2025
related version of root zone: 2025012901

FORMERLY NS.INTERNIC.NET

. 3600000 NS A.ROOT-SERVERS.NET.
A.ROOT-SERVERS.NET. 3600000 A 198.41.0.4
A.ROOT-SERVERS.NET. 3600000 AAAA 2001:503:ba3e::2:30
;
; FORMERLY NS1.ISI.EDU
. 3600000 NS B.ROOT-SERVERS.NET.
B.ROOT-SERVERS.NET. 3600000 A 170.247.170.2
B.ROOT-SERVERS.NET. 3600000 AAAA 2801:1b8:10::b
;
; FORMERLY C.PSI.NET
. 3600000 NS C.ROOT-SERVERS.NET.

```

With the previous configurations, the server is able to detect the network and connect, in addition to allowing other devices to connect and do things on it. In our case, we had a small problem with the configuration; this was caused by a directory error, preventing the server from initializing. To solve this, we put the DNS in a folder with this name and put it in the directory designated to `/etc/networks`. After that, we adjust, and then it should work.

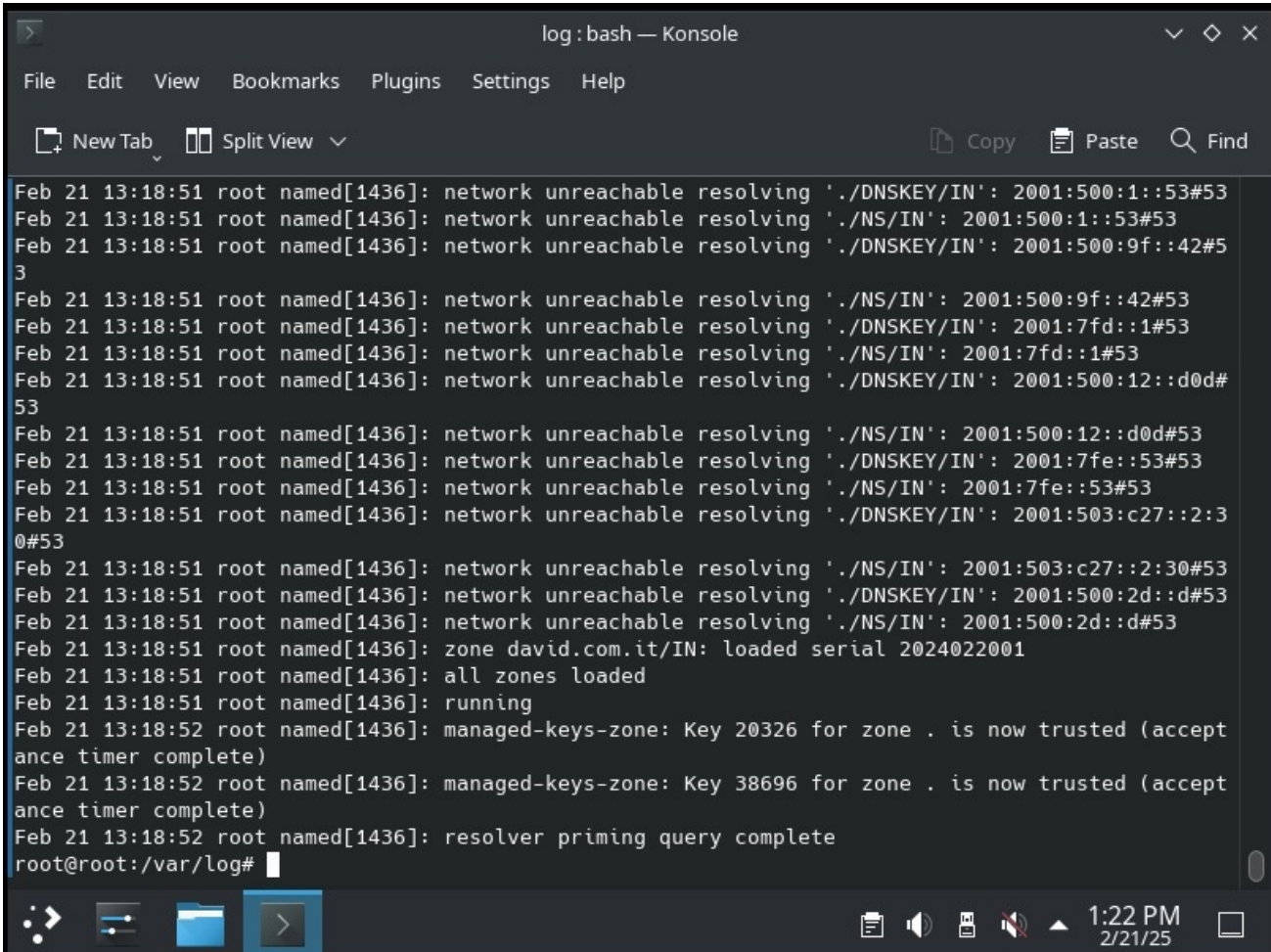
```
root@root:/etc/DNS# chmod 777 /etc/named.ca
root@root:/etc/DNS# chmod 777 /etc/named.conf
root@root:/etc/DNS# chmod 777 /etc/DNS/david.com.it.hosts
root@root:/etc/DNS# /etc/rc.d/rc.bind start
Starting BIND: /usr/sbin/named -u named
WARNING: named did not start.
Attempting to start named again: /usr/sbin/named -u named
FAILED: Sorry, a second attempt to start named has also failed.
There may be a configuration error that needs fixing. Good luck!
root@root:/etc/DNS#
```

Once everything works in theory, we can start the service by running the server file with the following command ``/usr/sbin/named -u start`` to Slackware y ``/etc/rc.d/named onestart`` to NetBSD. In this way, the DNS server will be active and ready to operate.

```
net# /etc/rc.d/named onestart
named already running? (pid=551).
net#
net# /etc/rc.d/named onerestart
Stopping named.
Feb 20 20:29:47 net named[551]: error writing NTA file for
not found
Feb 20 20:29:47 net named[551]: error writing NTA file for
t found
Starting named.
net# /etc/rc.d/named onestart
named already running? (pid=721).
net#
```

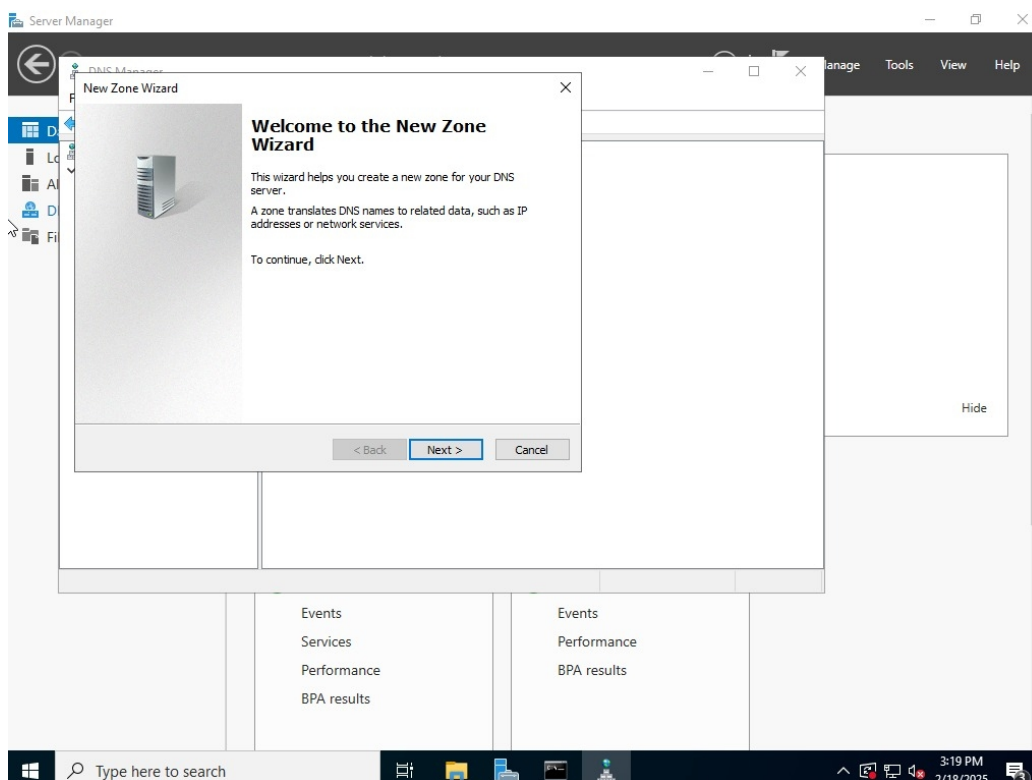
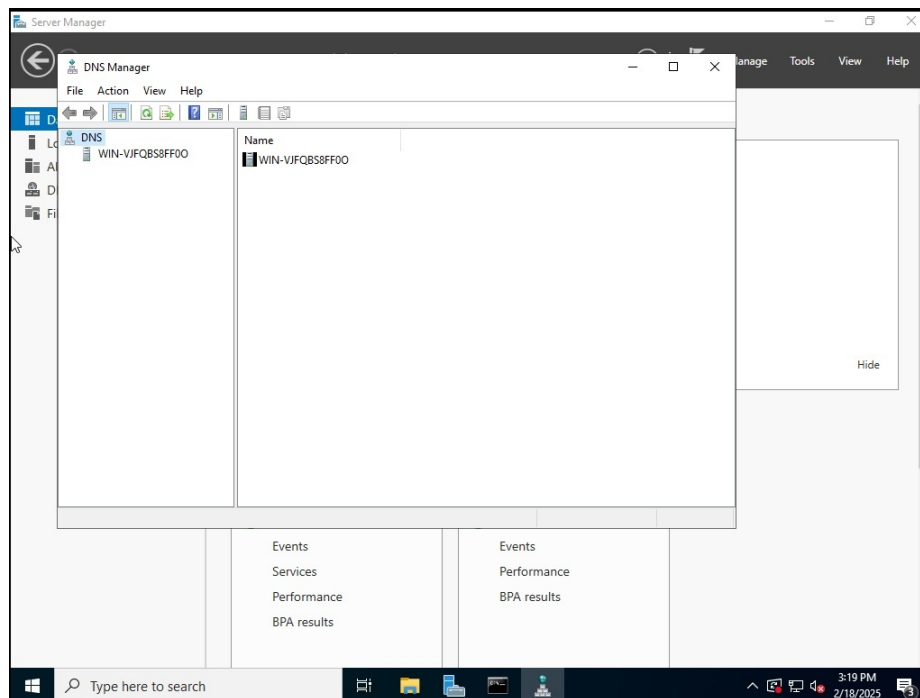
```
root@root:/etc# /etc/rc.d/rc.bind start
Starting BIND: /usr/sbin/named -u named
root@root:/etc#
```

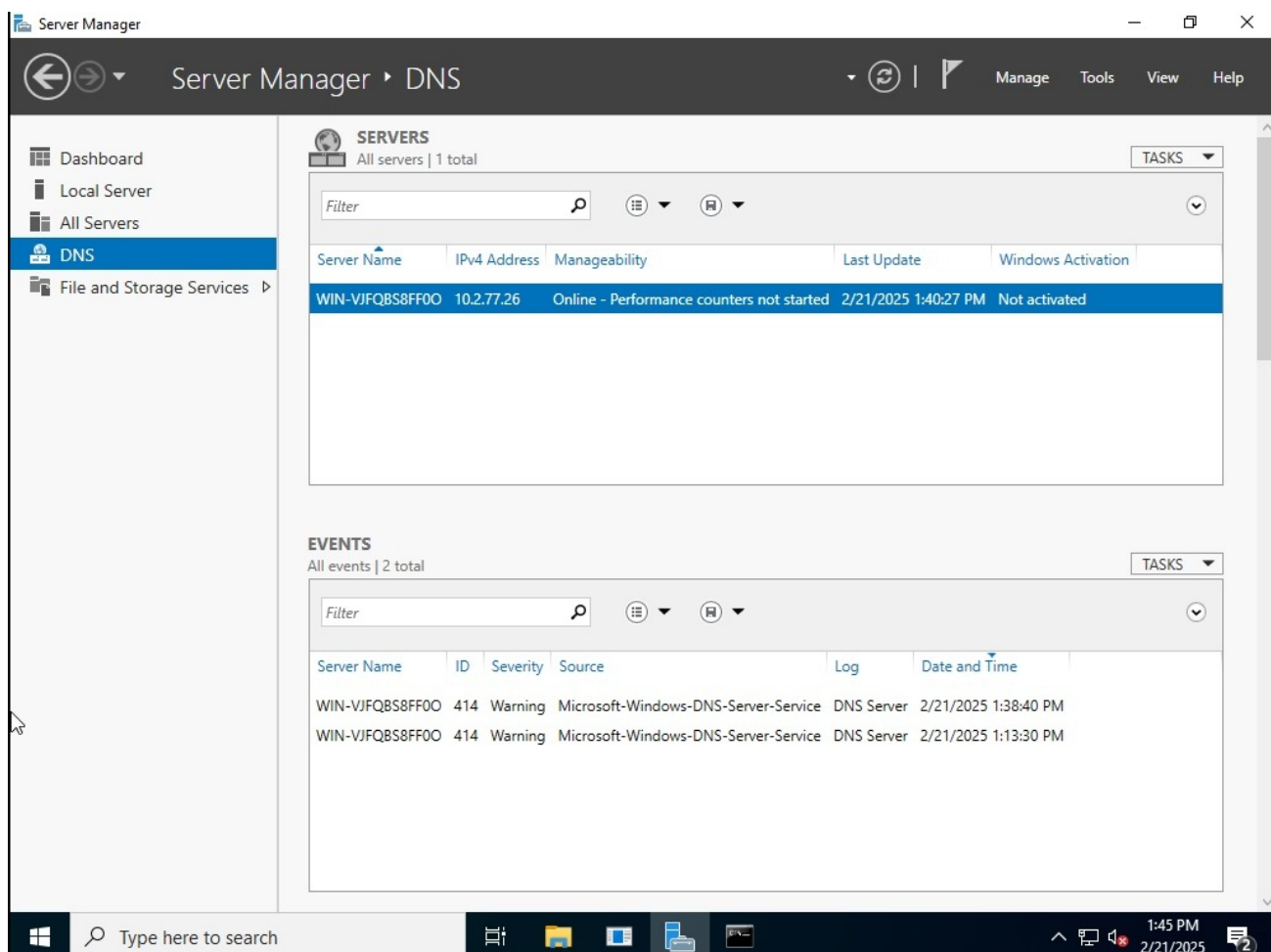
We can verify that it is mounted by looking at the system logs, so we use the `cat /var/log/messages | grep bind` command. In this, we see that the system is running perfectly (The errors are default configs that we deleted after this picture).



```
log : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:500:1::53#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:500:1::53#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:500:9f::42#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:500:9f::42#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:7fd::1#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:7fd::1#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:500:12::d0d#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:500:12::d0d#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:7fe::53#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:7fe::53#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:503:c27::2:30#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:503:c27::2:30#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './DNSKEY/IN': 2001:500:2d::d#53
Feb 21 13:18:51 root named[1436]: network unreachable resolving './NS/IN': 2001:500:2d::d#53
Feb 21 13:18:51 root named[1436]: zone david.com.it/IN: loaded serial 2024022001
Feb 21 13:18:51 root named[1436]: all zones loaded
Feb 21 13:18:51 root named[1436]: running
Feb 21 13:18:52 root named[1436]: managed-keys-zone: Key 20326 for zone . is now trusted (acceptance timer complete)
Feb 21 13:18:52 root named[1436]: managed-keys-zone: Key 38696 for zone . is now trusted (acceptance timer complete)
Feb 21 13:18:52 root named[1436]: resolver priming query complete
root@root:/var/log#
```

In the case of Windows, we use the Windows Server tool to connect to the DNS. To do this, we go to the area that has the DNS name and select it, and we choose the option to create a new zone for this service. Subsequently, we will have to configure the zone with the IP that our machine has assigned for the IPv4 connection, so that it appears active and ready to work.





Finally, we can check that the service is active by entering the `nslookup` command. Well, we tested it in general and tried to perform a verification with the university's DNS to prove that the service could connect to the Escuela's network.

```

root@root:/var/log# nslookup
g> google.com 10.2.65.1
Server:          10.2.65.1
Address:         10.2.65.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.78.46
Name:   google.com
Address: 64:ff9b::8efa:4e2e
> sS

```

Once we have the DNS ready, we are going to test them using different records of the protocol. To do this, we will use the `nslookup` command, which is explained in more detail in the video requested for this lab. But in general terms, it is a tool that will help us solve problems and obtain information related to DNS.

- NS (Domain File): Defines the authorized name servers for a domain, indicating where to look for information about it.

```
> set type=NS
> google.com
Server:          10.2.65.1
Address:         10.2.65.1#53

Non-authoritative answer:
google.com      nameserver = ns3.google.com.
google.com      nameserver = ns4.google.com.
google.com      nameserver = ns2.google.com.
google.com      nameserver = ns1.google.com.

Authoritative answers can be found from:
```

- Debug mode displays advanced details about DNS queries, including packets sent and received, response times, and records obtained.

```
@SANS_ISC C:\Demo>nslookup -d example.com. 8.8.8.8
-----
Got answer:
  HEADER:
    opcode = QUERY, id = 1, rcode = NOERROR
    header flags: response, want recursion, recursion avail.
    questions = 1, answers = 1, authority records = 0, additional = 0

  QUESTIONS:
    8.8.8.8.in-addr.arpa, type = PTR, class = IN
  ANSWERS:
    -> 8.8.8.8.in-addr.arpa
        name = dns.google
        ttl = 13759 (3 hours 49 mins 19 secs)

-----
Server: dns.google
Address: 8.8.8.8

-----
Got answer:
  HEADER:
    opcode = QUERY, id = 2, rcode = NOERROR
    header flags: response, want recursion, recursion avail.
    questions = 1, answers = 1, authority records = 0, additional = 0

  QUESTIONS:
    example.com, type = A, class = IN
  ANSWERS:
    -> example.com
        internet address = 93.184.215.14
        ttl = 792 (13 mins 12 secs)
```

- A and AAAA (Root Servers File): The A records contain IPv4 addresses and the AAAA records contain IPv6 addresses of the Internet root servers, essential for name resolution.

```

> set type=A
> google.com
Server:      10.2.65.1
Address:     10.2.65.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.78.46
>

```

- q=MX is used to query the MX records of a domain, showing the mail servers responsible for receiving emails.

```

Address: 142.250.78.46
> set q=MX
> google.com
Server:      10.2.65.1
Address:     10.2.65.1#53

Non-authoritative answer:
google.com   mail exchanger = 10 smtp.google.com.

Authoritative answers can be found from:
google.com   nameserver = ns4.google.com.
google.com   nameserver = ns2.google.com.
google.com   nameserver = ns1.google.com.
google.com   nameserver = ns3.google.com.
>

```

With everything shown, we can support the teacher and demonstrate the knowledge acquired. Furthermore, we saw that everything was working correctly and without problems.

OTHER USEFUL COMMANDS

In this section, we will write some shell scripts that allow us to automate processes and manage the services that are running in the virtual machine. In this way, we can link the practice carried out with the DNS by verifying that the system is really running the BIND packet.

The first script helps us automate tasks periodically. Well, in some cases, having a task that does not require our intervention is very useful. To do this, we will receive the parameters that will indicate the frequency in cron format and the command that we want to perform. Crontab is a package that allows us to perform this type of action, which is the reason the time format is given in the parameter.

The second script intends to display all the information related to the different services running on the virtual machine. In addition, you must allow user interaction to be able to unsubscribe, restart, or consult said services. To get a better idea, we have to perform the same operation as applications for Unix environments, such as `htop` or `btcp`. To do this, we are going to create the menu to perform all the actions requested of us. In this, we will have to obtain all the processes that run on the machine and be able to filter them if the user so wishes, in addition, we will be able to terminate it or restart it as needed.

The last script aims to navigate between directories based on a given directory. To do this, there is a limit to the number of files displayed on the screen and with a certain size as well. In this way, we can only see certain files that have the characteristics indicated as parameters. To do this, we request the parameters and go to the given directory, and then filter the files as indicated and show them to the user. To see this script, go to the attachments.

CONCLUTIONS

Through this lab, we successfully installed and configured multiple DNS servers using BIND on various operating systems, ensuring a functional and reliable domain name resolution system. We defined and tested different DNS records, including A, AAAA, NS, CNAME, and MX, verifying their correct configuration and ensuring proper resolution of both internal domain names and external addresses.

Additionally, we implemented Shell scripts to automate system tasks, manage processes, and analyze files, enhancing our ability to maintain and troubleshoot server environments efficiently. By utilizing tools like `nslookup`, we thoroughly tested DNS resolution, examined query responses, and validated the integrity of our DNS infrastructure by analyzing the records retrieved through the DNS protocol.

This experience provided valuable insights into network management, emphasizing the critical role of DNS services in IT environments. Furthermore, it highlighted the importance of automation in system administration, improving efficiency, reliability, and overall system security.

VIDEOS

In this section you will find the videos made for this laboratory, these are:

- Summary and explanation of `nslookup` command.

<https://youtu.be/qUFmH8IMI-k>

BIBLIOGRAFY

Axarnet. (2022, marzo 8). NSLOOKUP: Qué es y cómo usarlo en servidores DNS 【Guía】 .

Axarnet.es. <https://axarnet.es/blog/que-es-nslookup>

Chapter 1. Setting up and configuring a BIND DNS server. (s/f). Redhat.com. Recuperado el 23 de febrero de 2025, de https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_networking_infrastructure_services/assembly_setting-up-and-configuring-a-bind-dns-server_networking-infrastructure-services

Cómo configurar BIND como servidor DNS de red privada en Ubuntu 18.04. (s/f). Digitalocean.com. Recuperado el 23 de febrero de 2025, de <https://www.digitalocean.com/community/tutorials/how-to-configure-bind-as-a-private-network-dns-server-on-ubuntu-18-04-es>

DNS record types: Defined and explained. (s/f). Site24x7. Recuperado el 23 de febrero de 2025, de <https://www.site24x7.com/learn/dns-record-types.html>

nslookup: la herramienta para acceder a las entradas DNS. (s/f). IONOS Digital Guide. Recuperado el 23 de febrero de 2025, de <https://www.ionos.com/es-us/digitalguide/servidores/herramientas/nslookup/>

(S/f-a). Amazon.com. Recuperado el 23 de febrero de 2025, de <https://aws.amazon.com/es/route53/what-is-dns/>

(S/f-b). Cloudflare.com. Recuperado el 23 de febrero de 2025, de <https://www.cloudflare.com/learning/dns/dns-records/>